

VoiceChat-TTS: A Low-Latency Continuous Speech Synthesis Model for Interactive Agents

Edresson Casanova^{1,*,**}, Jaehyeon Kim^{1,*}, Mariana Graterol Fuenmayor¹, Shehzeen Hussain¹, Viacheslav Klimkov¹, Valentin Mendelev¹, Mikyas Desta¹, Paarth Neekhara¹, Piotr Zelasko¹, Chen Chen¹, Elena Rastorgueva¹, Ke Hu¹, Ankita Pasad¹, Xuesong Yang¹, Aya Alja'fari¹, Rajarshi Roy¹, Rohan Badlani¹, Jason Roche¹, Jason Li¹, Zhehuai Chen¹

NVIDIA Corporation

ecasanova@nvidia.com

Abstract

Spoken dialogue is a natural form of human-computer interaction, yet most speech language models remain limited to turn-based operation and lack real-time adaptability, such as user barge-in. Recent duplex speech-to-speech and speech-to-text models reduce latency by replacing multi-stage pipelines, but often compromise speech quality because accurate ASR, interruption handling, and high-fidelity synthesis must be optimized jointly. We propose VoiceChat-TTS, a low-latency, continuous, and streamable text-to-speech model for interactive agents. VoiceChat-TTS is driven directly by LLM text-token streams, supports explicit interruption via control tokens, and produces silence when no textual input is available. The model enables always-on, responsive speech generation while preserving modularity and high speech quality, and it supports mid-utterance interruptions without resetting the KV cache.

Index Terms: Speech synthesis, text-to-speech, streaming TTS, speech LLMs

1. Introduction

Spoken dialogue is a natural and intuitive modality for human-computer interaction. However, most existing speech language models remain constrained to turn-based operation and lack real-time adaptability, such as support for user barge-in. Recent duplex speech-to-speech (S2S) [1, 2, 3] and speech-to-text [4] models achieve low latency and simplified deployment by replacing traditional multi-stage pipelines that rely on separate automatic speech recognition (ASR), voice activity detection (VAD), and text-to-speech (TTS) components. ASR systems transcribe spoken audio into text, VAD detects the presence or absence of speech to guide downstream processing, and TTS generates waveform output from text input.

While end-to-end duplex models are highly promising, they often exhibit degraded speech synthesis quality. This degradation largely arises from the substantial data and modeling challenges involved in jointly optimizing accurate ASR, robust interruption handling, and high-quality speech generation within a single architecture.

Prior work on speech-to-speech (S2S) models has explored architectures that incorporate dedicated speech decoders connected to a shared backbone through latent representations [5, 6]. Although effective in reducing overall interaction latency, these approaches increase architectural complexity and can compromise the modularity and debuggability of traditional pipeline systems.

Recently, streaming TTS has made significant progress in reducing time-to-first-audio (TTFA) and enabling incremental speech generation. One early example is the streaming speech decoder of Audio Flamingo 3-Chat [7], which was designed for low-latency conversational speech generation by consuming streamable text inputs and producing speech tokens progressively. This design demonstrated that high-quality neural speech synthesis can be integrated into interactive speech-language systems without requiring the complete text response in advance. Subsequent streaming TTS systems further improved latency and alignment. For example, VoXstream [8] uses an incremental decoder-only Transformer with monotonic alignment and limited look-ahead to map incoming phonemes to acoustic tokens, while SpeakStream [9] employs a decoder-only architecture trained with interleaved text-and-speech sequences to absorb streaming text from large language models (LLMs). In addition, Qwen3-TTS [10] explores efficient text-to-speech alignment for real-time applications by directly ingesting LLM tokens to achieve very low TTFA while maintaining high synthesis quality and expressiveness.

However, these systems primarily address streaming generation within a single response. They do not natively model the continuous, always-on behavior required by full-duplex interactive agents, where the speech decoder must remain active across conversational time, generate silence when no agent text is available, and stop promptly in response to user barge-in. VoiceChat-TTS builds on this line of streaming speech decoders, particularly the Audio Flamingo 3-Chat speech decoder, and extends it with explicit silence modeling, interruption control tokens, and a unified training formulation for both single-turn and multi-turn conversational synthesis.

In this paper, we propose VoiceChat-TTS, a continuous, streamable, and low-latency text-to-speech model designed for interactive agents. VoiceChat-TTS is driven directly by large language model (LLM) text-token streams, supports explicit interruption through control tokens, and produces silence when no textual input is available. This design enables always-on, responsive speech generation while preserving modularity and high synthesis quality. Furthermore, VoiceChat-TTS is compatible with both duplex interaction frameworks and speech-to-speech pipeline systems, and avoids resetting the KV cache when generation is interrupted mid-utterance.

The main contributions of our work are as follows:

- We introduce VoiceChat-TTS, a continuous, streamable, and low-latency TTS model that directly consumes LLM text-token streams and generates silence when no agent text is available;
- We propose a reliable control-token-based interruption mech-

*These authors contributed equally.

**indicates the corresponding author.

anism that halts ongoing speech and transitions the output to silence during mid-utterance user barge-ins;

- We present a unified training strategy that combines high-quality single-turn TTS data with complex multi-turn conversational data while minimizing the distribution mismatch between the two settings;
- We demonstrate that VoiceChat-TTS achieves competitive speech quality relative to strong offline and streaming baselines while meeting the latency and interruption-handling requirements of interactive agents.

VoiceChat-TTS code is publicly available in NVIDIA NeMo Speech¹, and the model checkpoint is available on Hugging Face as part of NVIDIA-NemotronLabs-VoiceChat-11B².

2. VoiceChat-TTS Model

The proposed architecture builds directly upon the streaming speech decoder of Audio Flamingo 3-Chat [7], incorporating several modifications to support full-duplex interactions. The Audio Flamingo 3-Chat streaming speech decoder was designed for low-latency conversational use cases and real-time interactions. It relies on streamable inputs, allowing text to be processed incrementally without requiring the full sequence in advance, and produces streamable outputs by generating speech tokens progressively for immediate audio playback. However, fully duplex interactions impose additional operational requirements: the system must handle mid-sentence user interruptions and produce silence while the user is speaking to support natural turn-taking. To fulfill these requirements, we implement several modifications on top of the base Audio Flamingo 3-Chat streaming speech decoder architecture. Figure 1 shows an overview of the VoiceChat-TTS architecture.

Audio Codec: The audio codec backbone is a fully causal autoencoder composed entirely of convolutional layers, similar to that of [7]. In our work, we train the codec to compress 22 kHz waveforms at a frame rate of 12.5 Hz using 31-codebook Residual Vector Quantization (RVQ) tokens. In this configuration, each generated audio-token frame corresponds to an 80 ms waveform chunk. This frame rate is compatible with recent duplex speech-to-speech models such as [1, 2]. For efficient streaming codec inference, the network caches the inputs of the convolutional blocks only within their receptive field.

Text Tokenizer: In contrast to Audio Flamingo 3-Chat [7], we use the NVIDIA Nemotron Nano 2 subword tokenizer [11] and augment it with Beginning-of-Sequence (BOS) and interruption tokens. On the LibriTTS *test-clean* subset, the tokenizer produces text tokens at an average rate of 4.16 Hz, corresponding to approximately one text token for every three acoustic-token frames. During training, the text stream is right-padded to match the length of the acoustic sequence, enabling incremental processing throughout the interaction. A BOS token marks the beginning of each assistant turn, while an interruption token marks the point at which the model should stop speaking and transition to silence. We additionally introduce a one-token delay between the text and audio channels, with the text stream leading the audio stream. This provides limited linguistic look-ahead, ensuring that the model observes at least two subword tokens before generating the corresponding speech.

Character-Aware Subword Encoder: A prevalent issue in incremental text-to-speech models that consume LLM sub-

word tokens is the mismatch between LLM vocabularies and TTS training corpora; many subwords produced by the former are rare or absent in the latter. To mitigate this problem, we introduce a Character-Aware Subword Encoder. Each input subword is first converted into a sequence of characters, which is processed by a shallow Transformer encoder. We then average-pool the character-level outputs to obtain robust character-aware embeddings that can generalize to unseen subwords. Figure 2 shows an overview of the Character-Aware Subword Encoder. In addition, to improve pronunciation accuracy during incremental generation of long or fragmented words, we incorporate a dedicated embedding for subword-continuation tokens. This allows the model to process and synthesize multi-token linguistic units more cohesively.

Mixture of Gaussian Estimation Head: To accelerate generation of the deep RVQ token hierarchy, we integrate the Mixture of Gaussian head (MoGH) [12, 7] into the architecture. Instead of relying on a 31-step autoregressive decoding pipeline, this head iteratively unmask the RVQ tokens. In each iteration, it predicts the continuous embedding vector of the masked RVQ tokens using MoG estimation and subsequently quantizes the vector into progressively unmasked discrete tokens. This iterative refinement is controllable, and prior results indicate that 4 to 8 iterations are sufficient to achieve high-fidelity acoustic reconstruction [7].

Audio Prompt Conditioning: The Audio Flamingo 3-Chat streaming speech decoder does not employ explicit speaker-reference conditioning. Instead, it is trained on concatenated utterances from the same speaker and learns to infer speaker identity from preceding acoustic context. This approach can be less effective at the beginning of generation, when little or no speaker-informative speech is available, and in conversational settings, where the recent acoustic context may be dominated by silence while the user is speaking. To provide an explicit speaker cue, we condition VoiceChat-TTS on a 3-second reference audio prompt. During training, the corresponding audio tokens prefill the beginning of the acoustic sequence, and the loss over this prompt region is masked so that the model uses the prompt as conditioning context rather than learning to reconstruct it. During inference, the same 3-second prompt initializes the acoustic context before synthesis begins.

Boundary Embeddings and Gated Fusion: Since our training objective targets multi-turn conversations, we introduce learnable text embeddings for the BOS and interruption tokens. These embeddings provide explicit boundary signals that improve the smoothness of conversational turn-taking. We also address an architectural instability in the base decoder, where high-magnitude RVQ embeddings can cause mixed-precision overflow and destabilize early-layer activations. We mitigate this issue by applying a gated fusion mechanism between incoming text embeddings and acoustic speech embeddings, ensuring stable numerical scaling throughout the network.

The final VoiceChat-TTS model comprises 977M parameters in total, including a 778M-parameter Gemma 3-based streaming TTS module [13] and a 199M-parameter codec model.

3. Experiments

To train the VoiceChat-TTS model, we utilize a combination of standard single-turn text-to-speech corpora, synthetically generated multi-turn dialogues, and real-world conversational datasets. This diverse data mixture is essential for ensuring robust acoustic quality while simultaneously teaching the model

¹<https://github.com/NVIDIA-NeMo/Speech>

²<https://huggingface.co/nvidia/NVIDIA-NemotronLabs-VoiceChat-11B>

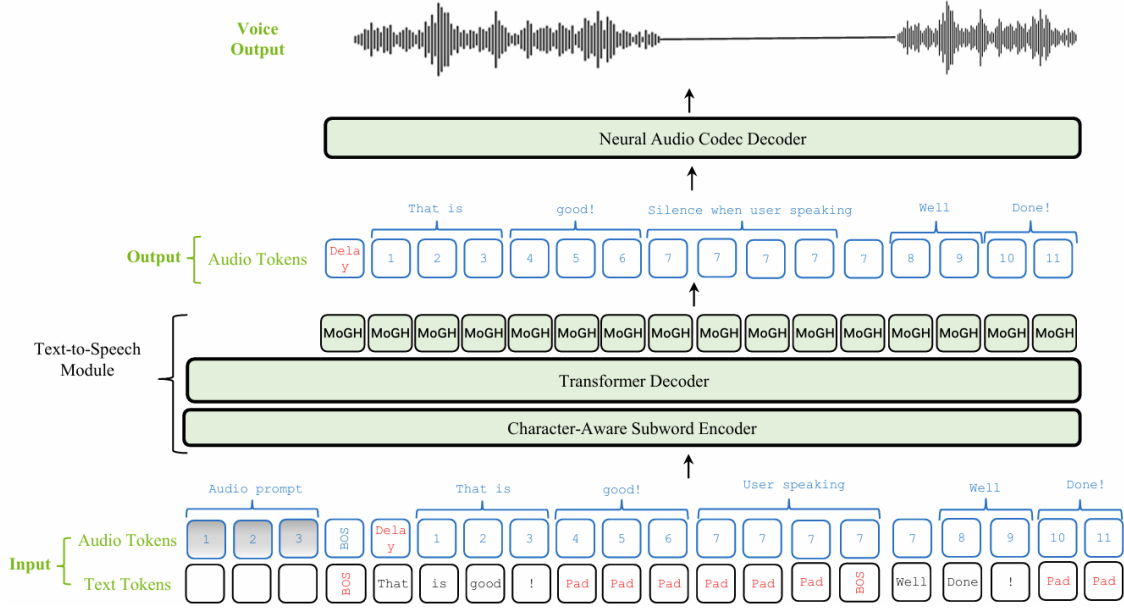


Figure 1: VoiceChat-TTS architecture overview.

complex interaction dynamics, such as turn-taking and interruption.

3.1. Single-turn data

To establish a robust foundation for acoustic generation, we leverage large-scale, standard text-to-speech corpora for the single-turn training phase. Specifically, we use approximately 70,159 hours of English speech sourced from the *train-clean-360* and *train-clean-100* subsets of LibriTTS [14], the original HiFiTTS corpus [15], an extended 36.7k-hour subset of HiFiTTS-2 [16], and a proprietary 62-hour dataset comprising two high-fidelity speakers. We additionally incorporate a 32k-hour internal dataset derived from high-quality, publicly available YouTube videos under the Creative Commons license.

A critical challenge in training duplex models is the distribution shift between single-turn generation, which typically begins speech synthesis immediately, and multi-turn interactions, which often begin with extended periods of silence while the user speaks. To mitigate this modality discrepancy, we apply a targeted data augmentation strategy to 50% of our single-turn data. We randomly prepend between 0.5 and 5.0 seconds of background silence to the beginning of the audio sequences, shifting the corresponding text alignments accordingly. This technique effectively simulates a multi-turn dialogue context where the agent is initially in a listening state. By bridging this gap, we prevent the model from trivially distinguishing between standard TTS tasks and duplex scenarios.

3.2. Multi-turn data

Synthetic datasets: To explicitly model complex duplex behaviors, we develop a curated synthetic multi-turn dataset comprising approximately 2.5k hours of speech. Millions of textual dialogue scripts are generated by open-source LLMs using NeMo Data Designer, with prompts tuned to cover a wide range of topics and situations. Each dialogue contains up to 15 turns and includes LLM-generated paralinguistic annotations. We then synthesize the scripts using a pipeline based on Chatterbox [17] and Koel-TTS [18], producing multi-speaker dialogues

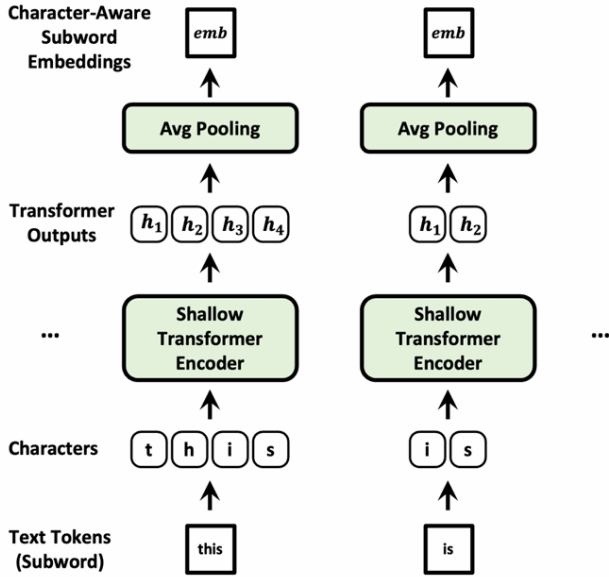


Figure 2: Character-Aware Subword Encoder architecture overview.

with speaker identities drawn from the LibriTTS training sets. Crucially, 50% of this synthetic data is designed to simulate overlapping speech and user barge-ins. This provides the alignment and explicit control tokens needed for the model to learn interruption handling. Furthermore, to expand phonetic coverage and improve robustness to out-of-vocabulary terms in conversational settings, we synthesize specialized multi-turn interactions containing complex words and proper names. In these interactions, each turn consists of four distinct words, systematically increasing the lexical diversity observed during duplex training.

Real conversation datasets: While synthetic data provides precise control over interaction timing and interruption labels, real conversational data is important for capturing natural dialogue dynamics. To this end, we incorporate unscripted data from the Fisher corpus alongside a small internal conversational dataset. These data allow the model to learn nuanced acoustic transitions, spontaneous backchannels, and natural turn-taking patterns that are difficult to fully reproduce through synthetic generation alone. Accurately transcribing paralinguistic events in spontaneous speech remains challenging; consequently, these datasets often contain untranscribed acoustic phenomena, such as laughter, sighs, and subtle backchannels, which the model must handle robustly.

3.3. Experimental setup

We train VoiceChat-TTS in two stages. First, we pretrain the model for 1.4M steps using only single-turn TTS data on 8 NVIDIA A100 GPUs. This stage provides a strong acoustic and linguistic foundation before introducing more complex conversational behavior. We then fine-tune the model for 200k steps on 32 NVIDIA H100 GPUs using the full training mixture, including single-turn TTS data, word and proper-name lists, real conversational speech, and synthetic duplex dialogues.

During fine-tuning, we use Lhotse [19] dynamic bucketing to construct batches containing approximately 240 seconds of audio. The training mixture consists of 40% single-turn TTS data, 15% word and proper-name list data, 10% real conversational data, and 35% synthetic duplex conversational data. To prevent any modality discrepancy during training, single-turn and multi-turn samples are processed identically. In both formats, text tokens are right-padded with special padding IDs within the text channel until they match the total temporal length of the current conversational turn [1, 5].

For all training stages, we use the AdamW optimizer with a learning rate of 4×10^{-5} .

3.4. Results and Discussion

We evaluate VoiceChat-TTS in terms of intelligibility, speaker similarity, and predicted overall speech quality, following the evaluation protocol of [18]. Intelligibility is measured using ASR-based character error rate (CER) and word error rate (WER), where transcriptions are produced by Parakeet-TDT [20]. Speaker similarity is measured as the cosine similarity between speaker embeddings extracted from the synthesized speech and the reference prompt audio using *TitaNet-Large* [21]. We denote this metric as speaker embedding cosine similarity (SECS). Overall speech quality is estimated using Squim-MOS [22]; we use it as a non-intrusive quality estimate rather than as a direct measure of naturalness.

At inference time, we use multinomial top- p sampling with $p = 0.95$, a classifier-free guidance (CFG) scale of 0.2, and a noise scale of 0.001 during Mixture-of-Gaussians (MoG) sam-

pling. We use 8 MoGH refinement iterations for all quality and latency evaluations. Because generation is stochastic, each experiment is repeated ten times and we report mean values with 95% confidence intervals. Fixed reference outputs are evaluated once.

For unseen-speaker evaluation, we use the full *test-clean* subset of LibriTTS [14]. For seen-speaker evaluation, we use five speakers observed during training. To evaluate the model in multi-turn settings, we construct synthetic multi-turn test conversations by grouping LibriTTS utterances from the same speaker. This setup allows us to measure whether synthesis quality remains stable as the number of generated turns increases.

Table 1 compares VoiceChat-TTS with strong offline and streaming TTS baselines. Conventional TTS systems, such as Chatterbox-TTS and Qwen3-TTS, achieve the best single-turn intelligibility, speaker similarity, and predicted overall speech quality. However, these models are optimized for standard single-response synthesis and are not designed for continuous duplex interaction, where the decoder must remain active across conversational time, generate silence when no agent text is available, and respond reliably to interruption control signals.

We also include the Audio Flamingo 3-Chat streaming speech decoder [7], the closest architectural baseline to VoiceChat-TTS. Compared with this baseline, VoiceChat-TTS substantially improves intelligibility and predicted overall speech quality. In the one-turn setting, VoiceChat-TTS reduces WER from 4.51% to 2.00% and increases Squim-MOS from 3.60 to 4.38, while achieving comparable speaker similarity. These results show that the complete VoiceChat-TTS system improves upon the Audio Flamingo 3-Chat streaming decoder while extending it to continuous multi-turn generation, silence modeling, and interruption-aware synthesis.

VoiceChat-TTS achieves competitive predicted overall speech quality while supporting the additional requirements of interactive agents. In the unseen-speaker setting, the model obtains a Squim-MOS of approximately 4.38 across all evaluated turn counts, close to the strongest baselines. Intelligibility also remains stable as the number of turns increases from one to four, with WER varying only from 2.00% to 2.20%. For seen speakers, all metrics remain similarly stable across multi-turn generation: CER and WER remain low, Squim-MOS stays around 4.36, and SECS remains within a narrow range from 0.778 to 0.789. This small seen-speaker similarity gap indicates that VoiceChat-TTS can preserve speaker identity consistently across multiple turns when the target speaker is well represented during training.

For unseen speakers, SECS decreases more noticeably as the number of generated turns increases, from 0.757 for one turn to 0.685 for four turns. This suggests that maintaining speaker identity over extended continuous generation remains more challenging in zero-shot or unseen-speaker settings, particularly when the model alternates between speech and silence states. Improving long-context speaker consistency for unseen speakers is therefore an important direction for future work, potentially through stronger prompt conditioning or explicit speaker-consistency objectives.

Finally, we compare VoiceChat-TTS with the PersonaPlex speech-to-speech (S2S) model [3]. PersonaPlex jointly predicts text and audio tokens. For this comparison, we use its outputs on the Smooth Turn Taking subset of Full-Duplex-Bench [24]. To isolate the contribution of the speech decoder, we keep the PersonaPlex-predicted text-token stream fixed and resynthesize it with VoiceChat-TTS. We preserve the original temporal align-

Table 1: Comparison between TTS models. VoiceChat-TTS is evaluated with different numbers of consecutive turns.

Model	Type	# Turns	Unseen Speakers				Seen Speakers			
			CER(%) ↓	WER(%) ↓	SECS ↑	Squim-MOS ↑	CER(%) ↓	WER(%) ↓	SECS ↑	Squim-MOS ↑
Ground Truth	-	-	0.48	1.40	0.830	4.457	-	-	-	-
Chatterbox-TTS [23]	Offline	-	0.45 ± 0.02	1.24 ± 0.02	0.887 ± 0.001	4.27 ± 0.003	-	-	-	-
Audio Flamingo 3-Chat [7]	Streaming	-	2.85 ± 0.33	4.51 ± 0.41	0.761 ± 0.002	3.60 ± 0.009	-	-	-	-
Qwen3-TTS-12Hz-1.7B [10]	Streaming	-	0.34 ± 0.02	1.01 ± 0.05	0.827 ± 0.001	4.45 ± 0.006	-	-	-	-
VoiceChat-TTS	Streaming	1	1.00 ± 0.10	2.00 ± 0.10	0.757 ± 0.004	4.380 ± 0.004	1.20 ± 0.10	2.40 ± 0.10	0.785 ± 0.001	4.365 ± 0.001
VoiceChat-TTS	Streaming	2	1.00 ± 0.10	1.90 ± 0.10	0.710 ± 0.001	4.377 ± 0.002	0.80 ± 0.10	1.90 ± 0.10	0.789 ± 0.005	4.358 ± 0.004
VoiceChat-TTS	Streaming	3	1.10 ± 0.10	2.10 ± 0.10	0.696 ± 0.001	4.381 ± 0.003	0.80 ± 0.10	1.80 ± 0.10	0.778 ± 0.001	4.362 ± 0.001
VoiceChat-TTS	Streaming	4	1.20 ± 0.20	2.20 ± 0.20	0.685 ± 0.005	4.376 ± 0.003	0.80 ± 0.10	1.80 ± 0.10	0.778 ± 0.001	4.359 ± 0.001

Table 2: Comparison between PersonaPlex and VoiceChat-TTS on the Smooth Turn Taking subset of Full-Duplex-Bench, using the same PersonaPlex-predicted textual content.

Model	CER(%) ↓	WER(%) ↓	Squim-MOS ↑
PersonaPlex [3]	4.06	5.00	4.094
PersonaPlex + VoiceChat-TTS	2.05 ± 0.78	2.42 ± 0.80	4.292 ± 0.007

ment of this stream, including PAD tokens during intervals in which the user is speaking. VoiceChat-TTS therefore runs continuously over the complete conversation timeline and is expected to generate silence during these intervals. CER and WER are computed from the full, untrimmed assistant output; consequently, any intelligible speech generated during an intended silence interval contributes ASR insertion errors. This setup evaluates both speech generation quality and the model’s ability to remain silent during user speech while preserving the same linguistic content and timing. Since speaker identities are not matched between systems, we omit speaker-similarity metrics from this comparison.

Table 2 shows that resynthesizing the PersonaPlex-predicted text with VoiceChat-TTS improves both intelligibility and predicted overall speech quality while preserving the original linguistic content and text-token timing. VoiceChat-TTS reduces CER from 4.06% to 2.05% and WER from 5.00% to 2.42%. Because these metrics are computed on the full, untrimmed assistant waveform, any intelligible speech produced during PAD-designated user-speech intervals contributes ASR insertion errors. The lower error rates therefore indicate both more intelligible synthesis of the intended content and limited intelligible speech leakage during intervals in which the model is expected to remain silent. VoiceChat-TTS also improves Squim-MOS from 4.094 to 4.292, indicating higher predicted perceptual quality. Overall, these results demonstrate the benefit of using VoiceChat-TTS as a modular, high-quality speech decoder in continuous S2S pipelines.

3.5. Interruption Evaluation

To evaluate whether VoiceChat-TTS responds correctly to explicit interruption signals, we construct an FDB-timed controlled-interruption benchmark based on the User Interruption subset of Full-Duplex-Bench (FDB). This evaluation should not be interpreted as an official FDB score. We use FDB only for the user audio and interruption-timing annotations; because the benchmark does not provide canonical assistant transcripts suitable for TTS-only evaluation, we generate controlled assistant responses with known reference text. Following the

Full-Duplex-Bench v1.5 stop-latency protocol [24], we insert the interruption token at the detected onset of the interrupting user speech and measure how rapidly the active assistant output transitions to silence.

Speech activity is detected independently in the original FDB user audio and the assistant-only output generated by VoiceChat-TTS. Both waveforms are resampled to 16 kHz and processed using the official FDB evaluation/get_timing.py implementation, which uses Silero VAD. Following the official configuration, adjacent user and assistant speech segments separated by less than 0.6 s and 0.5 s, respectively, are merged. We identify the interrupting user segment as the merged user segment with the greatest temporal overlap with the interruption interval provided in the FDB metadata. The onset of this segment defines the interruption time. Silero VAD detects active assistant speech at interruption onset in all 200 controlled examples.

We report three timing-based metrics. Interruption Obey Rate at 320 ms (IOR@320ms), is the percentage of examples in which the assistant speech segment active at interruption onset ends within 320 ms. FDB-v1.5-compatible Stop Latency is computed from the user–assistant overlap intervals returned by the official latency_stop_list; the table reports the mean after pooling these intervals across examples. Finally, Leakage@1s is the cumulative duration of detected assistant speech during the first second after interruption onset, averaged across examples.

We additionally report After-Interruption Character Error Rate (AI-CER) to evaluate whether the decoder can recover from an interruption and synthesize the subsequent assistant turn without resetting its KV cache. For each example, we isolate only the assistant-output segment corresponding to the controlled turn following the interruption, beginning at its known BOS timestamp and ending at the turn boundary. We transcribe this segment, compute CER against the corresponding reference text, and average the resulting CER values across examples. Audio between interruption onset and the subsequent BOS—including any residual overlap, leakage, or silence—is excluded. AI-CER therefore measures the intelligibility and content preservation of the subsequent assistant turn, complementing the stop-latency and leakage metrics rather than incorporating the interrupted portion of the output. The evaluation contains all 200 examples from the FDB User Interruption subset, and every example is processed for each reported metric.

We compare two inference settings. When *Force Silence* is false, the model must transition to silence solely through its learned response to the interruption token. When *Force Silence* is true, inference injects a fixed silence audio-token frame at positions aligned with the interruption token, deterministically steering the generated speech stream toward silence. To obtain

Table 3: *Effect of deterministic silence forcing on the FDB-timed controlled-interruption benchmark. Timing and leakage metrics are computed using the FDB v1.5 Silero VAD pipeline. AI-CER is computed only on the subsequent assistant turn and excludes the interruption-overlap interval. Values are means over ten stochastic runs; 95% confidence intervals are omitted for compactness.*

Force Silence	IOR@320ms (%) $\uparrow$	Mean stop latency (ms) $\downarrow$	Leakage@1s (ms) $\downarrow$	AI-CER (%) $\downarrow$
False	96.8	228.3	169.1	0.255
True	100.0	89.9	55.8	0.095

this frame, we encode a prolonged segment of pure silence with the codec and select the most frequently occurring 31-token codec frame.

Table 3 shows that VoiceChat-TTS learns a strong response to the interruption token even without deterministic enforcement, achieving an IOR@320ms of 96.8%. Enabling *Force Silence* increases IOR@320ms to 100.0%, reduces mean FDB-v1.5-compatible Stop Latency from 228.3 ms to 89.9 ms, and reduces Leakage@1s from 169.1 ms to 55.8 ms. AI-CER also decreases from 0.255% to 0.095%. Because AI-CER excludes all audio preceding the subsequent BOS, this result indicates that deterministic silence forcing does not impair the model’s ability to recover and generate the next assistant turn without resetting its KV cache. Overall, the results show that the learned interruption behavior is already reliable, while deterministic silence forcing provides stricter interruption compliance and substantially reduces residual speech after interruption.

3.6. Latency

We evaluate streaming latency using inter-token latency (ITL), defined as the wall-clock time required to produce one frame of acoustic tokens after receiving a streaming text update. VoiceChat-TTS operates at 12.5 Hz, so each acoustic-token frame corresponds to 80 ms of audio. Therefore, an ITL below 80 ms indicates that the acoustic-token predictor runs faster than real time.

For comparison, we use Qwen3-TTS-12Hz, which also operates at 12.5 Hz and is designed for low-latency streaming synthesis [10]. The Qwen3-TTS technical report provides latency numbers for its 12 Hz models, including first-packet latency, tokenizer decoding time, and steady-state LM time per packet. However, those measurements are reported using the authors’ internal vLLM engine on a “single typical computational resource”, without specifying a directly comparable GPU configuration. To obtain a more controlled comparison, we remeasure Qwen3-TTS-12Hz on the same RTX A6000 setup used for VoiceChat-TTS and report latency under the same measurement protocol. Both models are optimized using the vLLM-Omni framework.

The final VoiceChat-TTS model comprises 977M parameters in total, including a 778M-parameter Gemma 3-based streaming TTS module [13] and a 199M-parameter codec model. In contrast, the Qwen3-TTS-12Hz model used in this comparison is the 1.7B variant [10]. Thus, VoiceChat-TTS is smaller while targeting the same 12.5 Hz streaming regime.

Table 4 reports latency at concurrency levels 1 and 4. We separately measure acoustic-token ITL and codec decoding time. Acoustic-token ITL measures the time required by the neural decoder to produce the next frame of acoustic to-

Table 4: *Streaming latency comparison on an RTX A6000 GPU. ITL denotes the time to produce one frame of acoustic tokens at 12.5 Hz, corresponding to 80 ms of audio. Codec denotes the measured time to decode one acoustic-token frame into waveform audio.*

Model	Conc.	Params	Acoustic ITL (ms) $\downarrow$	Codec (ms) $\downarrow$	Total (ms) $\downarrow$
Qwen3-TTS-12Hz [10]	1	1.7B	15.44	4.90	20.34
VoiceChat-TTS	1	977M	7.16	2.46	9.62
Qwen3-TTS-12Hz [10]	4	1.7B	17.09	4.99	22.08
VoiceChat-TTS	4	977M	12.35	2.89	15.24

kens. Codec latency measures the time required to decode one acoustic-token frame into waveform audio, corresponding to 80 ms of speech. The total next-frame latency is computed as the sum of acoustic-token ITL and codec decoding latency.

Both codec decoders are optimized for persistent streaming inference. For the Qwen3-TTS codec, we retain the transformer KV state and causal convolution histories, cache transposed-convolution overlap, linearize streaming upsampling operations, and fuse Snake activations using native CUDA kernels with reusable FP16 buffers. For the VoiceChat-TTS codec, we use causal convolution caching, fuse the 31 codebook lookups into a single bit-exact CUDA kernel, replace channel-wise normalization sequences with native FP32 kernels, and cache the fixed inverse-STFT constants.

As shown in Table 4, VoiceChat-TTS achieves lower latency than Qwen3-TTS-12Hz under the same RTX A6000 measurement setup. At concurrency 1, VoiceChat-TTS reduces acoustic-token ITL from 15.44 ms to 7.16 ms, corresponding to a $2.1\times$ speedup. Its optimized codec is also faster, reducing decoding time from 4.90 ms to 2.46 ms. Overall, VoiceChat-TTS reduces next-frame latency from 20.34 ms to 9.62 ms, a $2.1\times$ improvement.

At concurrency 4, VoiceChat-TTS remains faster, reducing acoustic-token ITL from 17.09 ms to 12.35 ms and codec latency from 4.99 ms to 2.89 ms. This reduces total next-frame latency from 22.08 ms to 15.24 ms. All measured latencies are well below the 80 ms duration represented by one generated acoustic-token frame, indicating that both systems can run faster than real time. However, VoiceChat-TTS provides a larger latency margin for serving overheads such as batching, scheduling, network transport, playback buffering, and asynchronous codec execution, while using a smaller model.

4. Conclusions, Limitations, and Future Work

In this paper, we introduced VoiceChat-TTS, a low-latency, continuous, and streamable text-to-speech architecture designed specifically for interactive agents. Driven directly by LLM text-token streams, VoiceChat-TTS supports explicit mid-utterance interruptions through control tokens and generates silence when no textual input is available. The model enables always-on, responsive speech generation while preserving modularity and competitive speech quality. Furthermore, it can be integrated into both duplex interaction frameworks and speech-to-speech pipelines. Our evaluations demonstrate competitive intelligibility and predicted overall speech quality relative to strong offline and streaming baselines, together with effective interruption handling and low streaming latency. VoiceChat-TTS already serves as the speech decoder in the open-source

NVIDIA Nemotron VoiceChat-11B duplex S2S model, demonstrating that the proposed modular decoder can be integrated into a complete interactive speech system and support low-latency duplex interaction.

Current limitations include the absence of user-audio conditioning, which prevents the model from dynamically adapting its prosody to acoustic cues from the user, and the lack of controlled component-wise ablations. The proposed architectural changes were designed to address complementary limitations of the underlying streaming decoder, and preliminary experiments during model development indicated that their gains were cumulative. However, systematically quantifying the contribution of each component remains an important direction for future work. Because continuous duplex TTS is an emerging setting without standardized TTS-specific benchmarks, we also plan to develop broader evaluation protocols for long-horizon continuous generation, direct measurement of speech leakage during intended silence, and recovery across repeated interruptions.

5. References

- [1] K. Hu, E. Hosseini-Asl, C. Chen, E. Casanova, S. Ghosh, P. Zelasko, Z. Chen, J. Li, J. Balam, and B. Ginsburg, “Salm-duplex: Efficient and direct duplex modeling for speech-to-speech language model,” *arXiv preprint arXiv:2505.15670*, 2025.
- [2] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: a speech-text foundation model for real-time dialogue,” *arXiv preprint arXiv:2410.00037*, 2024.
- [3] R. Roy, J. Raiman, S.-g. Lee, T.-D. Ene, R. Kirby, S. Kim, J. Kim, and B. Catanzaro, “Personaplex: Voice and role control for full duplex conversational speech models,” *arXiv preprint arXiv:2602.06053*, 2026.
- [4] X. Lu, W. Xu, H. Wang, H. Zhou, H. Zhao, C. Zhu, T. Zhao, and M. Yang, “Duplexmamba: Enhancing real-time speech conversations with duplex and streaming capabilities,” in *CCF International Conference on Natural Language Processing and Chinese Computing*. Springer, 2025, pp. 62–74.
- [5] E. Casanova, C. Chen, K. Hu, A. Pasad, E. Rastorgueva, S. Lakshmi Narasimhan, S. Deng, E. Hosseini Asl, P. Zelasko, V. Mendelev, S. Ghosh, Y. Peng, Z. Chen, J. Li, J. Balam, V. Lavrukhin, and B. Ginsburg, “Open full-duplex voice agent with speech-to-speech language model,” in *2025 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2025.
- [6] J. Xu, Z. Guo, H. Hu, Y. Chu, X. Wang, J. He, Y. Wang, X. Shi, T. He, X. Zhu *et al.*, “Qwen3-omni technical report,” *arXiv preprint arXiv:2509.17765*, 2025.
- [7] A. Goel, S. Ghosh, J. Kim, S. Kumar, Z. Kong, S.-g. Lee, C.-H. H. Yang, R. Duraiswami, D. Manocha, R. Valle *et al.*, “Audio flamingo 3: Advancing audio intelligence with fully open large audio language models,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [8] N. Torgashov, G. E. Henter, and G. Skantze, “Voxstream: Full-stream text-to-speech with extremely low latency,” *arXiv preprint arXiv:2509.15969*, 2025.
- [9] R. H. Bai, Z. Gu, T. Likhomanenko, and N. Jaitly, “Speakstream: Streaming text-to-speech with interleaved data,” *arXiv preprint arXiv:2505.19206*, 2025.
- [10] H. Hu, X. Zhu, T. He, D. Guo, B. Zhang, X. Wang, Z. Guo, Z. Jiang, H. Hao, Z. Guo *et al.*, “Qwen3-tts technical report,” *arXiv preprint arXiv:2601.15621*, 2026.
- [11] NVIDIA, “Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.14444>
- [12] J. Kim, T. Moon, K. Lee, and J. Cho, “Efficient generative modeling with residual vector quantization-based tokens,” in *International Conference on Machine Learning*. PMLR, 2025, pp. 30 609–30 630.
- [13] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, L. Rouillard, T. Mesnard, G. Cideron, J. bastien Grill, S. Ramos, E. Yvinec, M. Casbon, E. Pot, I. Penchev, G. Liu, F. Visin, K. Kenealy, L. Beyer, X. Zhai, A. Tsitsulin, R. Busa-Fekete, A. Feng, N. Sachdeva, B. Coleman, Y. Gao, B. Mustafa, I. Barr, E. Parisotto, D. Tian, M. Eyal, C. Cherry, J.-T. Peter, D. Sinopalnikov, S. Bhupatiraju, R. Agarwal, M. Kazemi, D. Malkin, R. Kumar, D. Vilar, I. Brusilovsky, J. Luo, A. Steiner, A. Friesen, A. Sharma, A. Sharma, A. M. Gilady, A. Goedeckemeyer, A. Saade, A. Feng, A. Kolesnikov, A. Bendebury, A. Abdagic, A. Vadi, A. György, A. S. Pinto, A. Das, A. Bapna, A. Miech, A. Yang, A. Paterson, A. Shenoy, A. Chakrabarti, B. Piot, B. Wu, B. Shahriari, B. Petrini, C. Chen, C. L. Lan, C. A. Choquette-Choo, C. Carey, C. Brick, D. Deutsch, D. Eisenbud, D. Cattle, D. Cheng, D. Paparas, D. S. Sreepathihalli, D. Reid, D. Tran, D. Zelle, E. Noland, E. Huizenga, E. Kharitonov, F. Liu, G. Amirkhanyan, G. Cameron, H. Hashemi, H. Klimczak-Plucińska, H. Singh,

- H. Mehta, H. T. Lehari, H. Hazimeh, I. Ballantyne, I. Szpektor, I. Nardini, J. Pouget-Abadie, J. Chan, J. Stanton, J. Wieting, J. Lai, J. Orbay, J. Fernandez, J. Newlan, J. yeong Ji, J. Singh, K. Black, K. Yu, K. Hui, K. Vodrahalli, K. Greff, L. Qiu, M. Valentine, M. Coelho, M. Ritter, M. Hoffman, M. Watson, M. Chaturvedi, M. Moynihan, M. Ma, N. Babar, N. Noy, N. Byrd, N. Roy, N. Momchev, N. Chauhan, N. Sachdeva, O. Bunyan, P. Botarda, P. Caron, P. K. Rubenstein, P. Culliton, P. Schmid, P. G. Sessa, P. Xu, P. Stanczyk, P. Tafti, R. Shivanna, R. Wu, R. Pan, R. Rokni, R. Willoughby, R. Vallu, R. Mullins, S. Jerome, S. Smoot, S. Girgin, S. Iqbal, S. Reddy, S. Sheth, S. Pöder, S. Bhatnagar, S. R. Panyam, S. Eiger, S. Zhang, T. Liu, T. Yacovone, T. Liechty, U. Kalra, U. Evci, V. Misra, V. Roseberry, V. Feinberg, V. Kolesnikov, W. Han, W. Kwon, X. Chen, Y. Chow, Y. Zhu, Z. Wei, Z. Egyed, V. Cotruta, M. Giang, P. Kirk, A. Rao, K. Black, N. Babar, J. Lo, E. Moreira, L. G. Martins, O. Sanseviero, L. Gonzalez, Z. Gleicher, T. Warkentin, V. Mirrokni, E. Senter, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, Y. Matias, D. Sculley, S. Petrov, N. Fiedel, N. Shazeer, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, J.-B. Alayrac, R. Anil, Dmitry, Lepikhin, S. Borgeaud, O. Bachem, A. Joulin, A. Andreev, C. Hardin, R. Dadashi, and L. Hussenot, “Gemma 3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.19786>
- [14] H. Zen, V. Dang, R. Clark, Y. Zhang, R. J. Weiss, Y. Jia, Z. Chen, and Y. Wu, “LibriTTS: A corpus derived from librispeech for text-to-speech,” *INTERSPEECH*, 2019.
- [15] E. Bakhturina, V. Lavrukhin, B. Ginsburg, and Y. Zhang, “Hi-Fi Multi-Speaker English TTS Dataset,” in *INTERSPEECH*, 2021.
- [16] R. Langman, X. Yang, P. Neekhara, S. Hussain, E. Casanova, E. Bakhturina, and J. Li, “Hifits-2: A large-scale high bandwidth speech dataset,” *arXiv preprint arXiv:2506.04152*, 2025.
- [17] resemble-ai, “Chatterbox: Sota open-source tts,” <https://github.com/resemble-ai/chatterbox>, 2025, gitHub repository.
- [18] S. S. Hussain, P. Neekhara, X. Yang, E. Casanova, S. Ghosh, R. Fejgin, M. T. Desta, R. Valle, and J. Li, “Koel-tts: Enhancing llm based speech generation with preference alignment and classifier free guidance,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 21 230–21 245.
- [19] P. Želasko, D. Povey, J. Trmal, S. Khudanpur *et al.*, “Lhotse: a speech data representation library for the modern deep learning ecosystem,” *arXiv preprint arXiv:2110.12561*, 2021.
- [20] H. Xu, F. Jia, S. Majumdar, H. Huang, S. Watanabe, and B. Ginsburg, “Efficient sequence transduction by jointly predicting tokens and durations,” in *International Conference on Machine Learning*. PMLR, 2023. [Online]. Available: <https://huggingface.co/nvidia/parakeet-tdt-1.1b>
- [21] N. R. Koluguri, T. Park, and B. Ginsburg, “Titanet: Neural model for speaker representation with 1d depth-wise separable convolutions and global context,” in *ICASSP*. IEEE, 2022. [Online]. Available: https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/titanet_small
- [22] A. Kumar, K. Tan, Z. Ni, P. Manocha, X. Zhang, E. Henderson, and B. Xu, “Torchaudio-squim: Reference-less speech quality and intelligibility measures in torchaudio,” in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [23] Resemble AI, “Chatterbox-TTS,” <https://github.com/resemble-ai/chatterbox>, 2025, gitHub repository.
- [24] G.-T. Lin, S.-Y. S. Kuan, Q. Wang, J. Lian, T. Li, S. Watanabe, and H.-y. Lee, “Full-duplex-bench v1. 5: Evaluating overlap handling for full-duplex speech models,” *arXiv preprint arXiv:2507.23159*, 2025.